

template = delete

Document Number: P2041R1
Date: 2021-03-09
Author: David Stone (davidfromonline@gmail.com)
Audience: Evolution Working Group (EWG)

Summary

We can delete function overloads. This paper proposes extending the functionality to variable templates and class templates. This paper does not propose allowing deleting non-template classes or variables.

```
// Delete primary variable template, allow specializations
template<typename>
int x = delete;
template<>
int x<int> = 5;

// Delete variable specialization
template<typename T>
auto y = T();
template<>
auto y<int> = delete;

// Delete primary class template, allow specializations
template<typename>
struct s = delete;
template<>
struct s<int> {
};

// Delete class specialization
template<typename>
struct t {
};

template<>
struct t<int> = delete;
```

Changes since R0

R0 suggested only allowing `= delete` for variable templates. This version also supports deleting class templates. Furthermore, it looks like the ability to delete deduction guides was approved long ago, but never made it into the working draft. This paper also adds a list of other types of entities and a discussion of why to not delete those.

Conceptual basis

`= delete` is valuable when:

- there is a single name that means multiple things
- some of those things should not exist, but the rules of C++ declarations make them exist

For things like non-template classes, rather than defining a class as deleted, you could just not bother defining it to begin with for the same effect. This is why it is not proposed to allow deleting any other entity -- there is no motivation for the work.

variable templates

Motivating example

```
template<typename T>
auto max_value = delete;

template<std::unsigned_integral T>
inline constexpr auto max_value<T> = T(-1);

template<std::signed_integral T>
inline constexpr auto max_value<T> = static_cast<T>(max_value<std::make_unsigned_t<T>> / 2);

template<typename T>
concept maxable = requires { max_value<T>; };

static_assert(maxable<int>);
static_assert(!maxable<std::string>);
```

Recommended solution

Variable templates are the natural way to define a value trait: it is explicitly a value templated on some type. The alternative is the more old-school approach of defining a trait in terms of a class template with a `static constexpr auto value` member, and then define a variable template that removes the boilerplate. This ends up duplicating your API and leads to natural questions of which version a user wants for reading the value vs. specializing the value. A SFINAE-friendly approach to removing the primary template of a variable template is the last missing piece to making variable templates a direct expression of users' intent, and using the syntax `= delete` is the natural way to express this intent.

Attempt to do this today: constraints

A first attempt to do this today might look like this:

```

template<typename T> requires false
int max_value;

template<std::unsigned_integral T>
inline constexpr auto max_value<T> = T(-1);

template<std::signed_integral T>
inline constexpr auto max_value<T> = static_cast<T>(max_value<std::make_unsigned_t<T>> / 2);

```

This immediately runs into a few compiler errors:

```

<source>:8:23: error: variable template partial specialization is not more specialized than the primary template [-Winvalid-partial-specializ
inline constexpr auto max_value<T> = T(-1);
                        ^
<source>:5:5: note: template is declared here
int max_value;
    ^
<source>:11:23: error: variable template partial specialization is not more specialized than the primary template [-Winvalid-partial-specializ
inline constexpr auto max_value<T> = static_cast<T>(max_value<std::make_unsigned_t<T>> / 2);
                        ^
<source>:5:5: note: template is declared here
int max_value;
    ^
2 errors generated.

```

Attempt to do this today: always invalid expression

```

template<typename T>
auto max_value = not defined(max_value<T>);
// Silly code credit to Richard Smith; note that this is equivalent to
// auto max_value = sdghdfksljadslfkjslfk<T>();

template<std::unsigned_integral T>
inline constexpr auto max_value<T> = T(-1);

template<std::signed_integral T>
inline constexpr auto max_value<T> = static_cast<T>(max_value<std::make_unsigned_t<T>> / 2);

template<typename T>
concept maxable = requires { max_value<T>; };

static_assert(!maxable<void>);

```

Rather than evaluating to false, the maxable<void> expression is a hard error.

```

<source>:5:22: error: use of undeclared identifier 'defined'
auto max_value = not defined(max_value<T>);
                        ^
<source>:16:30: note: in instantiation of variable template specialization 'max_value' requested here
concept maxable = requires { max_value<T>; };
                        ^
<source>:16:30: note: in instantiation of requirement here
concept maxable = requires { max_value<T>; };
                        ^~~~~~
<source>:16:19: note: while substituting template arguments into constraint expression here
concept maxable = requires { max_value<T>; };
                        ^~~~~~
<source>:18:16: note: while checking the satisfaction of concept 'maxable<void>' requested here
static_assert(!maxable<void>);
                ^~~~~~
1 error generated.

```

Attempt to do this today: extern incomplete

```

struct incomplete;

template<typename>
extern incomplete max_value;

template<std::unsigned_integral T>
inline constexpr auto max_value<T> = T(-1);

template<std::signed_integral T>
inline constexpr auto max_value<T> = static_cast<T>(max_value<std::make_unsigned_t<T>> / 2);

template<typename T>
concept maxable = !std::is_same_v<decltype(max_value<T>), incomplete>;

static_assert(!maxable<void>);

```

This all works correctly. Unfortunately, this, too, has drawbacks. Users of this variable template need to know that they cannot just rely on normal substitution failures and thus they cannot use it directly in a requires clause as part of a larger test for expression validity. Instead, all users need to do a test with is_same_v. Moreover, there does not appear to be a way to prevent users from passing around a reference or pointer to an instantiation of the primary template. This appears to be the best we can do under current rules.

Chosen syntax

We can delete functions, including the primary template of a function, with the behavior needed for variable templates. The following code is valid today:

```

template<typename T>
void foo() = delete;

template<std::unsigned_integral T>

```

```

void foo() {
}

template<>
void foo<int>() {
}

template<typename T>
concept fooable = requires { foo<T>(); };

static_assert(fooable<unsigned>);
static_assert(fooable<int>);
static_assert(!fooable<void>);

```

The `= delete` syntax has become an intuitive way for users to say that some portion of an otherwise generated interface should not actually be considered valid code, and that using a deleted overloads or specializations should be a substitution failure and interact usefully with SFINAE. Deleting a variable template (either the primary template or an explicit specialization) is a natural evolution.

A potential concern is that this syntax could possibly be ambiguous with operator delete, but this does not appear to be the case. Existing code that would look most like this falls into one of two forms:

```

template<typename T>
T thing = operator delete;
// Requires that T is something like void (*)(void *)

```

and

```

template<typename T>
T thing = (delete ptr, T());

```

In other words, using delete as a pointer to function requires the operator keyword before it, and using delete as an expression requires an operand after it.

class templates

This paper proposes allowing deleting the primary template of a class and specializations of class templates.

Given that we have the ability to leave a class incomplete, what advantages does this give us? Conceptually, an incomplete type has two potential meanings: 1) This is an opaque type that you can manipulate only by reference, or 2) This template instantiation is "wrong" or does not meaningfully exist. Deleting a class template gives us a mechanism that prevents forming pointers or references (or otherwise using the name), similar to how you cannot take the address of a deleted function overload. It also gives us the conceptual distinction, allowing incomplete class types to be forward declarations that can be used indirectly until it is completed at some later point (possibly in a different translation unit), while deleted classes cannot be used at all.

Note that here "class" is meant in the general sense including unions.

Motivating cases

```

#include <functional>

auto wrapper(auto && generate) {
    return generate();
}

void f(std::function<int> const & g) {
    wrapper(g);
}

```

Today, this gives a compiler error in the call to generate in the body of wrapper, but the bug actually occurs in the declaration of f. The user meant to write `std::function<int>()`, but accidentally wrote `std::function<int>`. It would be nice if `std::function` could explicitly state that the primary template is not defined and will never be defined, and thus it is always an error to use it.

Class template syntax

```

// Delete class primary template
template<typename>
struct t = delete;

template<>
struct t<int> {
};

// Delete class specialization
template<typename>
struct u {
};

template<>
struct u<int> = delete;

```

Deduction guides

EWG already approved treating the lack of ability to delete deduction guides as a defect against C++17, but this appears to have gotten lost. This paper proposes allowing deleting deduction guides as defined in [P0091](#), but takes no position on which standard that should apply against.

Things not proposed to allow deleting

The following entities cannot be deleted today, and are not proposed to be deletable:

- variable
- member variable
- class
- enum
- enum class
- enumerator
- typedef
- alias
- alias template
- module
- namespace
- namespace alias
- using directive
- using declaration
- asm
- concept

There doesn't seem to be much difference between users getting an error message saying "This thing you mentioned doesn't exist" vs "This thing you mentioned is deleted" for any things on this list.

Non-proposal: Deleting things as a general name-hiding mechanism

Some people have suggested allowing deleting variables as a way to remove access to a variable defined in wider scope. For example:

```
int x;

void f() {
    int x = delete;
    ++x; // Error, x is deleted
}
```

There are three reasons this is not proposed.

1. This doesn't follow the model of a "variable". An actual variable has a call to its destructor at the end of the scope. The implicit call to the destructor would try to access a deleted entity, which would not work.
2. The justification for this is weak.
3. Something similar can be achieved today with deleted functions that covers most of the use cases.

```
namespace deleted {
void x() = delete;
}

int x;

void f() {
    using deleted::x;
    ++x; // error
}
```